

7장 개념정리: BERT · BART · ELECTRA

7장 개념 정리: BERT · BART · ELECTRA

 파이토치 트랜스포머를 활용한 자연어 처리와 컴퓨터비전 심층학습 — 7장 (PDF pp. 406~451)

목차

1. BERT
 2. BERT 사전 학습 방법
 3. BERT 특수 토큰
 4. BERT 입력 임베딩 구조
 5. BERT 미세 조정
 6. BART
 7. BART 노이즈 기법
 8. BART 미세 조정
 9. ROUGE 평가 지표
 10. ELECTRA
 11. 모델 비교 요약
-

1. BERT

개요

- Bidirectional Encoder Representations from Transformers
- 구글(Google), 2018년 발표
- 트랜스포머 인코더만 사용 → 양방향(Bidirectional) 구조

핵심 아이디어: 양방향 언어 모델

기존 언어 모델 (단방향):
나는 [학교에] → 학교에 = 이전 단어(나는)만 참조

BERT (양방향):
나는 [MASK] 갔다 → MASK = 이전(나는) + 이후(갔다) 모두 참조

주요 특징

특징	설명
양방향 문맥 파악	전후 단어를 모두 참조해 더 정확한 의미 이해
전이 학습 (Transfer Learning)	대규모 데이터 사전 학습 후 다운스트림 작업에 재사용
적은 데이터로 높은 정확도	사전 학습 덕분에 소량 데이터로 미세 조정 가능
학습 시간 단축	사전 학습 가중치 재사용으로 학습 비용 절감

모델 크기

버전	인코더 층	헤드 수	임베딩 차원	매개변수
BERT-base	12	12	768	1.1억
BERT-large	24	16	1024	3.4억

2. BERT 사전 학습 방법

1) MLM (Masked Language Modeling) — 마스킹된 언어 모델링

입력: "I'm learning [MASK]"

목표: [MASK] = "PyTorch" 예측

- 입력 문장의 약 15% 토큰을 무작위 마스킹
- 양방향 문맥 활용 → 마스킹된 단어 예측
- 별도 레이블 불필요 → 자기 지도 학습(Self-supervised)

마스킹 전략 세부: - 선택된 토큰의 80%: [MASK] 로 대체 - 10%: 무작위 단어로 대체 - 10%: 원래 단어 유지
(모델이 어느 위치를 예측해야 할지 모르게 함)

2) NSP (Next Sentence Prediction) — 다음 문장 예측

입력: [CLS] 문장A [SEP] 문장B [SEP]

목표: 문장B가 문장A 다음에 오는 문장인지 (IsNext / NotNext) 예측

- 두 문장의 관계를 학습 → QA, NLI(자연어 추론) 성능 향상
- 50%: 실제 연속 문장 쌍 (IsNext)
- 50%: 무작위 문장 쌍 (NotNext)

3. BERT 특수 토큰

토큰	역할
[CLS]	문장 시작 / 분류 태스크에서 전체 문장 표현
[SEP]	두 문장 구분 / 문장 끝 표시
[MASK]	마스킹된 위치 표시 (MLM 학습용)
[PAD]	시퀀스 패딩
[UNK]	어휘 사전에 없는 단어

BERT 입력 형식

[CLS] 문장-1 [SEP] 문장-2 [MASK] [SEP]

4. BERT 입력 임베딩 구조

BERT의 입력 임베딩은 세 가지 임베딩의 합산:

입력 임베딩 = 토큰 임베딩 + 위치 임베딩 + 세그먼트 임베딩

임베딩	설명
토큰 임베딩 (token_embeddings)	각 토큰을 고정 차원 벡터로 매핑
위치 임베딩 (position_embeddings)	시퀀스 내 각 토큰의 위치 정보
세그먼트 임베딩 (token_type_embeddings)	두 문장 중 어느 문장인지 구분 (문장 A=0, 문장 B=1)

BERT 모델 구조

```

bert
├─ embeddings
│   ├── word_embeddings: Embedding
│   ├── position_embeddings: Embedding
│   ├── token_type_embeddings: Embedding
│   └─ LayerNorm + Dropout
├─ encoder (인코더 블록 × 12)
│   └─ 각 블록: 셀프 어텐션 + FFN
└─ pooler
    └─ [CLS] 토큰 벡터 → Linear + Tanh
classifier: Linear ← 미세 조정 레이어
  
```

Pooler: [CLS] 토큰 벡터에 선형 변환 + Tanh 적용 → 문장 분류에 사용

5. BERT 미세 조정

분류 태스크

- [CLS] 토큰 벡터 → Dropout → Linear → 예측
- 활용 예: 감성 분석, 문장 분류

토큰 태스크

- 모든 토큰 벡터 활용
- 활용 예: 개체명 인식(NER), 품사 태깅

활용 사전학습 모델

다국어 지원 (104개 언어)

```
BertTokenizer.from_pretrained("bert-base-multilingual-cased")
```

do_lower_case=False: 대소문자 유지

최적화 함수: AdamW

- Adam + **가중치 감쇠(Weight Decay)** 결합
- 엡실론(ϵ) 매개변수: 학습률을 0으로 나누는 것 방지용 분모 보정값
- 대규모 매개변수 모델의 안정적 학습에 적합

6. BART

개요

- Bidirectional Auto-Regressive Transformer
- 메타(Meta) FAIR 연구소, 2019년 발표
- BERT 인코더 + GPT 디코더 결합 구조
- 노이즈 제거 오토인코더(Denoising Autoencoder) 방식으로 사전 학습

사전 학습 방식

입력 (노이즈 추가된 문장) → [인코더] → [디코더] → 원본 문장 복원

- BERT처럼 마스킹 + GPT처럼 다음 단어 예측 모두 수행
- 입력에 큰 제약 없이 다양한 노이즈 기법 적용 가능

BART vs 트랜스포머 구조 차이

항목	트랜스포머	BART
인코더-디코더 어텐션	모든 인코더 층 ↔ 모든 디코더 층	인코더 마지막 층 ↔ 각 디코더 층
장점	-	정보 전달 최적화, 메모리 절약

공유 임베딩 (Shared)

- 인코더와 디코더가 **동일한 임베딩 가중치** 공유
- 인코더-디코더 연결 강화

BART 모델 구조

```

shared: Embedding (공유 임베딩)
encoder
├─ embed_tokens (shared)
├─ embed_positions
└─ layers (× 6)
    └─ 인코더 블록 (셀프 어텐션 + FFN)
        └─ layernorm_embedding
decoder
├─ embed_tokens (shared)
├─ embed_positions
└─ layers (× 6)
  
```

- └ 디코더 블록 (셀프 어텐션 + 교차 어텐션 + FFN)
- └ layernorm_embedding

7. BART 노이즈 기법

사전 학습 시 입력에 다양한 방식으로 노이즈를 추가해 더 풍부한 언어 지식 습득

기법	설명	효과
토큰 마스킹 (Token Masking)	BERT MLM과 동일, 일부 단어를 [MASK] 로 치환	문맥 이해, 중요 정보 추출
토큰 삭제 (Token Deletion)	일부 토큰 완전 삭제 (위치도 맞춰야 함)	불필요 정보 필터링, 일반화 성능 향상
문장 순열 (Sentence Permutation)	마침표(.) 기준으로 문장 순서 섞기	문장 간 관계 학습
텍스트 채우기 (Text Infilling)	연속 토큰들을 묶어 구간으로 만들고 하나의 [MASK] 로 대체	누락 단어 예측, 입력 이해 강화

텍스트 채우기 상세: - 구간 길이: 포아송 분포($\lambda=3$)로 결정 - 길이 0 = 해당 위치에 추가 마스크 토큰 삽입 - 모델이 연속 마스크 토큰의 개수도 예측해야 함

8. BART 미세 조정

태스크별 입력 구조

태스크	인코더 입력	디코더 입력	예측
문장 분류	입력 문장	입력 문장 (동일)	디코더 마지막 토큰 은닉 상태 → 선형 분류기

태스크	인코더 입력	디코더 입력	예측
토큰 분류	입력 문장	입력 문장 (동일)	디코더 각 시점 은닉 상태 → 토큰별 분류
문장 요약	본문 (긴 텍스트)	요약문	디코더 출력 → 요약 문장 생성
기계 번역	소스 언어	타겟 언어	디코더 출력 → 번역 문장

BERT vs BART 분류 차이: BERT는 [CLS] 토큰 벡터 사용, BART는 전체 입력과 어텐션 연산이 적용된 마지막 토큰 은닉 상태 사용

BART 강점 태스크

- 추상적 질의응답 (Abstractive QA)
- 문장 요약 (Text Summarization) — 사전 학습 방식과 유사해 특히 뛰어난 성능
- BERT가 해결하지 못한 **생성 작업** 가능

사용 모델 클래스

```
BartForConditionalGeneration.from_pretrained("facebook/bart-base")
```

9. ROUGE 평가 지표

요약 모델 평가에 사용하는 지표 (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE-N

$$\text{ROUGE-N 재현율} = \frac{\text{생성 문장과 정답 문장에 등장한 N-gram 수}}{\text{정답 문장에 등장한 N-gram 수}}$$

$$\text{ROUGE-N 정밀도} = \frac{\text{생성 문장과 정답 문장에 등장한 N-gram 수}}{\text{생성 문장에 등장한 N-gram 수}}$$

예시 (ROUGE-1, 유니그램)

정답 문장: 대한민국은 16강에 진출했다
생성 문장: 대한민국은 8강에 진출하지 못했다

공통 유니그램: {대한민국, 은, 강, 에, 진출} → 5개
정답 유니그램: 7개
생성 유니그램: 9개

ROUGE-1 재현율 = $5/7 \approx 0.71$
ROUGE-1 정밀도 = $5/9 \approx 0.56$

ROUGE 변형

지표	설명
ROUGE-1	유니그램(1-gram) 기반
ROUGE-2	바이그램(2-gram) 기반
ROUGE-L	최장 공통 부분 수열(LCS) 기반
ROUGE-W	가중치 LCS 기반

설치 및 사용

```
pip install evaluate rouge_score absl-py
```


모델	발표	구조	방향	사전학습 방법	강점 태스크
BART	2019 (Meta)	인코더 +디코더	양방향 +단방향	노이즈 제거 오토인코더	요약, 번역, 생성
ELECTRA	2020 (구 글)	인코더 +판별기	양방향	대체 토큰 탐지(RTD)	분류 (효율적)

한눈에 보는 사전학습 비교

BERT: 나는 [MASK] 갔다 → "학교에" 예측 (15%만 학습)
 GPT: 나는 학교에 → "갔다" 예측 (단방향)
 BART: 나는 [MASK] [삭제] → 원본 문장 복원 (다양한 노이즈)
 ELECTRA: 나는 회사에 갔다 → "회사에" = Replaced 탐지 (100% 학습)